BeMaMaJa

Group Project Report – INFO4271 Modern Search Engines

Matthias Vetter, Marcel John, Janis Weller, Bernhard Fraidel

https://github.com/BeMaMaJa-Search-Engine/BeMaMaJa-Search-Engine.git

Abstract

This project presents the implementation of a search engine from scratch. It starts by implementing a web crawler that uses a given seed set to retrieve web pages. To avoid crawling undesirable websites, a blacklist is used, and the crawler respects the robots.txt protocol. To improve performance, the crawler employs multiprocessing. For indexing, the retrieved pages are tokenized, stop words are removed, and the remaining words are stemmed before constructing the inverted index. The retrieval stage is based on the BM25 algorithm with an integrated spelling correction mechanism. For re-ranking, we apply field boosts, link counts and PRF and combine them with the BM25 score. The frontend features a clean and modern user interface. Additionally, it provides AI-powered features that offer users supplementary information about the retrieved search results.

1 Data Collection and Crawling

1.1 Corpus scope and seeds

The scope of our project is to build a search engine that indexes English-language websites related to Tübingen. To create an initial set of seed URLs for the crawler, we first collected a list of around 50 keywords that represent different topics connected to the city. These keywords included areas such as attractions, the university, student life, hospitals, culture, and other relevant subjects. It was important to choose keywords that cover a wide range of topics so that the resulting corpus is as diverse as possible.

For each keyword, we searched Google using the query "Tübingen + keyword" and manually recorded the first 10 search results. After collecting all URLs, duplicate links were removed. Starting from around 50 keywords, this process resulted in a final seed list containing 449 unique URLs, which served as the starting point for our crawler.

We had one manual adjustment of the seed list and put https://scells.me/ into it manually, bringing our total to 450.

1.2 Crawler design

During development, we implemented four different crawler versions. To evaluate them, each version was allowed to run for about 15 hours under the same conditions. Based on the results, we selected the best-performing version for our final large-scale crawl. Since the results chapter compares these versions, we briefly describe each of them here.

Version 1 followed the crawler architecture introduced in the lecture. It used a prioritized frontier to decide which URLs to visit next and respected website politeness by enforcing crawl delays and setting a custom user agent. Instead of relying on the language information provided by websites, we used the `langdetect` Python library to ensure that only English pages were crawled. The crawler also respected `robots.txt` files by avoiding blocked paths, following crawl delays, and caching each `robots.txt` file to avoid downloading it repeatedly. Duplicate content was not removed during crawling but later during preprocessing, meaning that the crawler temporarily stored a larger `raw_pages.json` file in exchange for faster crawling. To improve crawling speed, the crawler used multiple worker threads.

Version 2 was based on Version 1 but improved the detection of Tübingen-related pages. Instead of a simple regular expression, it used a larger regular expression that also matched common spelling variations and OCR-like errors, such as "Tueeebingen" or "t123bingen". The idea was to identify more relevant pages even if the city name was written incorrectly or if stylistic choices of the author changed the spelling of Tübingen (such as in a blog post).

Version 3 focused mainly on improving performance. While testing Version 1, we noticed that sorting the prioritized frontier became increasingly expensive as it grew larger. Since the main thread was responsible for maintaining the sorted frontier, it often blocked the worker threads while performing these operations. We therefore tried a different approach. Instead of one globally prioritized frontier, we split it into two frontiers: a high-priority frontier containing links extracted from pages that were related to Tübingen, and a low-priority frontier containing links from unrelated pages.

The frontier itself was also redesigned. Instead of storing every URL in one large data structure, it became a list of domain objects. Each object contained a queue of URLs for a single domain together with information such as the last access time to enforce politeness delays. To select the next URL, the crawler randomly chose a domain that was ready to be visited and then processed the next URL from that domain. This naturally prevented the crawler from spending most of its time on very large websites. For example, even if the University of Tübingen website contained thousands of links while another domain only contained a handful, both domains had an equal chance of being selected.

This design comes with clear trade-offs. The main advantage is performance. Even with millions of URLs in the frontier, selecting the next domain depends only on the number of unique domains rather than the total number of URLs. Operations within each domain queue are also very efficient, allowing the main thread to spend much less time managing the frontier and more time supplying work to the worker threads. The downside is that we lose fine-grained prioritization within a domain. Once a domain is selected, URLs are processed in the order they were discovered instead of globally prioritizing the most relevant URL. This means that a URL containing the word "Tübingen" might be processed after less relevant URLs from the same website. Although this is a disadvantage, the large performance improvement outweighed the loss of prioritization. In our experiments, Version 3 crawled roughly four to five times as many pages as Versions 1 and 2 during the same 15-hour runtime.

Version 4 built directly on the ideas from Version 3 and introduced two different types of worker threads called Horses and Explorers. Horses behaved exactly as before by selecting a random domain that was ready to be crawled and processing its next URL. Explorers, on the other hand, searched for ready domains with only a small number of remaining URLs and processed those first. Since the Horses would eventually reach these smaller domains anyway, this change was not expected to have a large effect on the overall crawl. Instead, the Explorers were intended to discover new domains slightly earlier and improve exploration of the web.

A blacklist for domains was also implemented, containing https://archive.org.

1.3 Results

We wrote ourselves some tools to analyse our crawlers. These tools can be found in *scripts/crawler_analysis.py*. These are a few stats aswell as figures.

The following table shows the general performance of a crawler-version after about 10 hours of runtime. Impressively enough the number of unique domains of the base version and regex version (both using the single frontier+prioritization) cannot be matched by any of the coming crawler versions. On the other side, the effective throughput of new pages is nearly 4x the first version of our crawler.

A fair comparrison between V2 and V4 shows, that we gain about 3.57x the amount of pages while loosing about 4% of unique domains. A tradeoff we decided to go with. (WHich is also not taking into account that the more links are in our frontier the slower the V1 and V2 crawlers will become).

Crawler	**Runtime (minutes)**	**# domains**	**# raw pages**	**Pages per minute**
Base (V1) (blue)	632	471	5187	8.21
Regex (V2) (orange)	583	484	5201	8.92
Harsh Frontier (V3) (red)	635	416	20001	31.5
Horses & Explorers (V4) (purple)	628	452	20000	31.85

Table 1: Crawler Version comparison on the metrics of number of raw pages and number of unique domains.

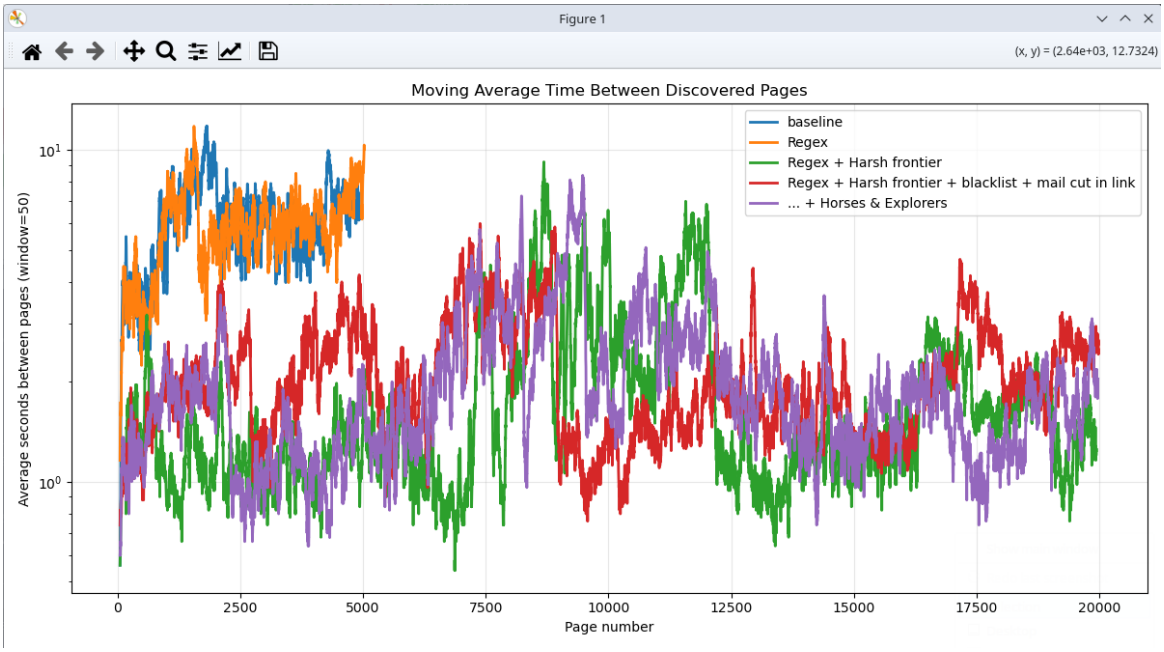

Figure 1: Average time between finding new pages with a window of 50 to smooth the curve. (Lower is better)

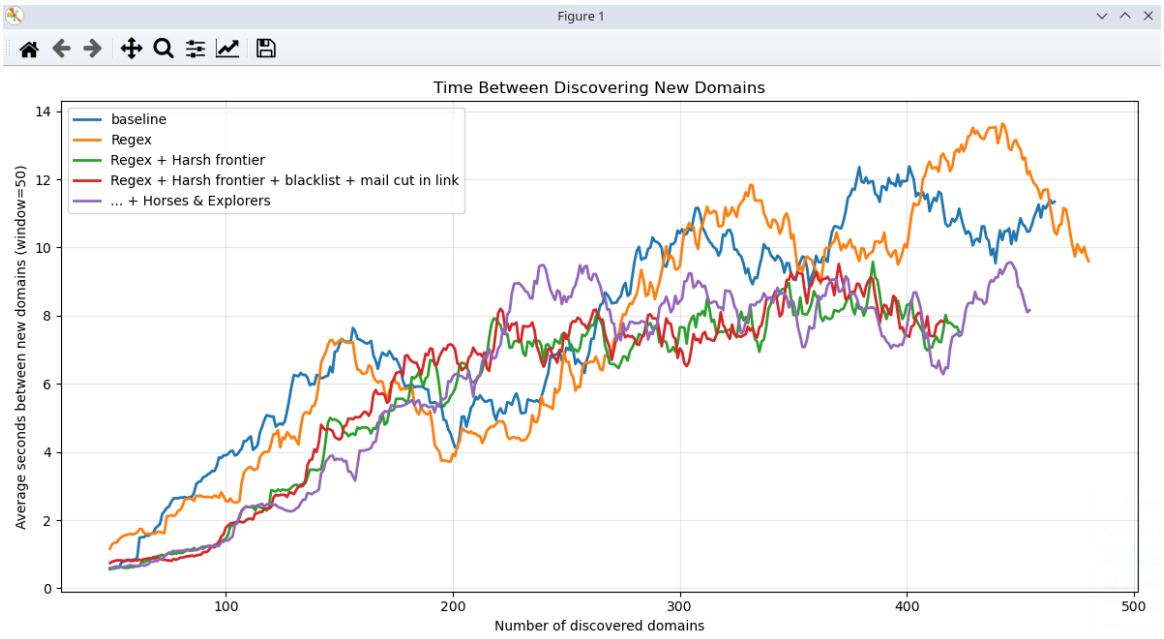

Figure 2: Time between discovering new domains with a sliding window of 50 to smooth the curve. (Lower is better)
Also visible here is the total amount of domains found.

Letting the crawler run for 43h16m14s resulted in a total of 75000 raw pages (reminder that there will be duplicates in here, this step is still done in preprocessing) from a total of TODO unique domains.

2 Preprocessing and Index Construction

After crawling, the collected pages were preprocessed before building the search index. First, duplicate pages were removed, which reduced the dataset by about 30

To further improve the quality of the dataset, language detection was run again to discard documents that were not in English. Pages containing a high proportion of non-Latin characters were also removed. Finally, Unicode normalization was applied to ensure a consistent text representation before giving the data to the next step.

2.1 Shared preprocessing contract

2.2 Manual inverted index

2.3 Build process and validation

Table 2: Index statistics and build characteristics.

Measure	Value
Indexed documents	
Vocabulary size	
Average body length	
Documents with outgoing links	
Index size / build time	

2.4 Engineering justification

3 Retrieval and Re-ranking

3.1 Manual BM25 first stage

BM25 is used as the first-stage ranking as it is a standard algorithm in practice. It computes the Inverse Document Frequency (IDF) to give rare words a higher value. Since the search engine is Tübingen based the IDF for Tübingen is boosted by the factor 2. The Term Frequency (TF) is used to give words that appear more often a higher value. The final BM25 score is obtained by summing the contribution of each query term. The parameter b controls the strength of the document length normalization. The value $b = 0.75$ is chosen since it is the default value in the literature. The parameter k_1 controls the rate of term saturation. Here, the value $k_1 = 1.2$ is chosen since it is the default value in the literature. The BM25 score for a document d and a query q is computed as follows:

$$\text{BM25}(d, q) = \sum_{t \in q} IDF(t) \frac{tf_{t,d}(k_1 + 1)}{tf_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}.$$

[1]

3.2 Query processing

When processing a query, first for the BM25 step and the spelling correction, the existing cache is used to load the JSON file. The file contains the documents as well as the vocabulary inside those documents. Here, this vocabulary already gets separated into buckets for the later spelling correction. This usage of the cache allows for a significant speedup since the JSON file is only loaded and parsed once. Once the data is loaded, the tokens get preprocessed with the same preprocessing as when indexing the documents. Once this is done, the spelling of the query terms is checked. When checking a term, the algorithm first checks if the term exists in the vocabulary, since the vocabulary is built using the indexed pages. The vocabulary uses the stemmed terms. If a page contains that term, it can be identified as a correct spelling. In this case, no spelling correction is needed. If, however, the term is not in the vocabulary, it is assumed that the word contains a spelling mistake, and the algorithm tries to fix this spelling mistake. In order to avoid changing the word too much, if the length of the word is less than five, only one character can be changed; otherwise, up to two. To speed up the process, the buckets are used. Those buckets are sorted by the first letter and the length of the word. That way, only valid candidates have to be evaluated. For a further speed up the algorithm allows for an early exit if it is no longer possible to find a better term. Next, the Levenshtein Edit Distance [2] is computed. This value indicates how similar two words are. $d(i, j)$ is the Levenshtein Edit Distance between the prefixes $s_{1\ldots i}$ and $t_{1\ldots j}$. s_i and t_j are the i-th and j-th characters of the words.

$$d(i,j) = \begin{cases} \max(i,j), & \text{if } \min(i,j) = 0, \\ \min \begin{cases} d(i-1,j) + 1, \\ d(i,j-1) + 1, \\ d(i-1,j-1) + \begin{cases} 0, & \text{if } s_i = t_j, \\ 1, & \text{if } s_i \neq t_j, \end{cases} \end{cases} & \text{otherwise.} \end{cases}$$

This is used to find the best candidate. If two words have the same Edit Distance, their frequency is used as a tie-breaker, since the assumption is that a user is more likely to search for a more frequently used term. If the Edit Distance for the best candidate is above the number of characters that are allowed to be changed, the word will not be changed to avoid changing the query too much. After a term has been corrected the word gets stemmed again to ensure the resulting term is the same as without the spelling mistake, thus the search results are also identical. After this, the BM25 score is computed, and the result is then used by the reranking algorithm.

3.3 Second-stage innovation

The first improvement is adding field boosts to the score. The field boost increases the score when query terms appear in the title or the headings, since this indicates that the query terms are relevant to the given document. The values are normalized by the number of Queries in order to prevent the score to increase because of repeating terms. After computing the field boost for both fields, the two values are combined using a $7 : 3$ ratio, since the title is more important than the headings. For the computation, the following formulas are used, where Q is the

query, T represents the title tokens, and H represents the heading tokens.

$$\text{TitleCoverage} = \frac{|Q \cap T|}{|Q|}$$

$$\text{HeadingCoverage} = \frac{|Q \cap H|}{|Q|}$$

$$\text{FieldBoost} = 0.7 \cdot \text{TitleCoverage} + 0.3 \cdot \text{HeadingCoverage}$$

3.4 Domain diversification

To prevent the result list from being dominated by a single website, we added a soft domain-diversification step after reranking. The first result from each domain keeps its full score. Further results from the same domain are progressively penalized using a factor of 0.93 per repetition. The penalty is limited to a minimum factor of 0.75. We intentionally avoided a strict domain limit because several pages from the same website may still be highly relevant. This approach improves result variety while ensuring that relevant documents remain within the top 100 results used for evaluation.

3.5 Diagnostic evaluation

Table 3: Ablation or diagnostic comparison.

Configuration	Query 1	Query 2	Interpretation
BM25 only			
BM25 + innovation			

4 Frontend and Hosting

Frontend innovations. A good search engine should not only return relevant links, but also help users understand and work with the results. We added several innovations, mainly focused on controllable AI summaries for individual pages and clear insight into why each result appears at its position.

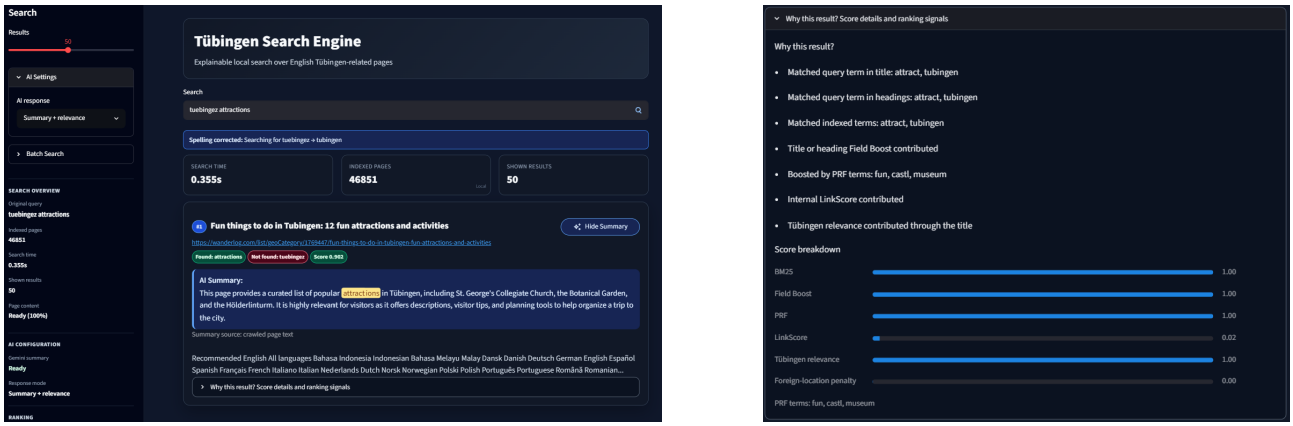

Figure 1. Search interface with AI controls, result scores, and an expanded ranking explanation.

4.1 Interactive interface

AI summaries. Each result card offers an *AI Summary* button backed by the Gemini API. *Summary + relevance* connects the page summary to the query, *Summary only* gives a neutral overview, and *Custom focus* accepts a short prompt, for example about opening hours. A custom animation is shown during generation. Here, page content means the crawled body text from the raw-page data; it is used only for AI summaries, not for retrieval or ranking. When unavailable, the snippet is used, and if the available text is too short, the page can be fetched live. If Gemini is unavailable or not configured, a local extractive summary is shown instead. Summaries are cached to avoid unnecessary API calls.

Explainable ranking. Each result card directly shows its final ranking score. A *Found* badge lists the exact matched query terms, while *Not found* lists the remaining terms. Applied spelling corrections are shown above the results so users know which terms were used for retrieval. For further detail, the expandable *Why this result?* view reports textual match evidence and a normalized breakdown of every active component: BM25, Field Boost, PRF, LinkScore, Tübingen relevance and foreign-location penalty.

Controls and fallback modes. Users can select between 5 and 100 displayed results. The sidebar reports search time, index size, active ranking components, data source (Supabase or local), Gemini status (ready or not configured), and page-content loading progress (crawled body text for AI summaries). The required batch file can be uploaded there and is returned as a downloadable `results.tsv` after validation and retrieval. Runtime data normally comes from a private Supabase bucket. Starting with `--local` disables remote access, and local index files provide a fallback if Supabase cannot be used. These options keep the service stable: if one feature or external service fails, the rest of the search continues to work so the user notices as little of the failure as possible.

4.2 Hosting and resource management

When a user opens the hosted version, DuckDNS resolves the domain to the public IP of the Oracle VM. Nginx, the reverse proxy, forwards the browser request to Streamlit on the same VM. Streamlit serves the interface and performs retrieval using the in-memory index. It downloads the compressed index and raw-page shards from Supabase Storage and sends summary requests to Gemini. The API key remains securely on our server; running locally requires a separate key.

Index caching. Caching the loaded index was our largest runtime improvement: it remains in RAM across Streamlit reruns and is reused for all searches. This reduced search times from about ten seconds to usually below one second.

Startup memory. Separately, as the index and raw pages grew, combining downloaded shards in RAM caused a large startup peak. They are now streamed into a disk cache first, while raw pages are processed gradually and only document IDs and body text needed for summaries are retained. This lowers peak RAM, makes search available earlier, and improves scalability without keeping unnecessary page data in memory.

References

[1] Stephen Robertson and Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009.

[2] Vladimir I. Levenshtein. *Binary Codes Capable of Correcting Deletions, Insertions and Reversals*. Soviet Physics Doklady, 10(8):707–710, 1966.